

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 1 — GIT & DOCKER TECHNICAL ASSIGNMENT

Technical Report

Assigned Problem: Question 19

Intelligent Public Health Surveillance and Disease Outbreak Prediction System

Course Code	CSA10	Course Title	Software Engineering
CO Assessed	CO3	Assessment	Assessment Tool 1 — Git & Docker Technical Assignment
Weightage	20%	Total Marks	25
Student Name	SHEIK MOHAMED	Reg. No.192511145	_____
Date	_____	Duration	60 minutes

Code of Conduct

I SHEIK MOHAMED(192511145)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

Industry Context

Public health agencies require continuous disease surveillance to detect outbreaks and implement preventive measures in a timely manner. Artificial Intelligence (AI), Big Data Analytics, Geographic Information Systems (GIS), Internet of Things (IoT), and cloud-based healthcare platforms together enable early detection of infectious diseases and support evidence-based public health decision-making. Modern healthcare ecosystems generate enormous volumes of structured and unstructured data from hospitals, diagnostic laboratories, wearable sensors, and citizen-reported symptoms, and this data can be harnessed to identify disease patterns well before they escalate into full-scale outbreaks.

The convergence of these technologies allows public health authorities to move from a reactive model of disease management to a proactive, predictive model — one where potential outbreaks are flagged days or weeks in advance, resources are pre-positioned in anticipated hotspot regions, and the general public is informed through timely alerts.

Globally, initiatives such as India's Ayushman Bharat Digital Mission (ABDM), the US CDC's National Syndromic Surveillance Program, and the WHO's Epidemic Intelligence from Open Sources (EIOS) demonstrate that governments increasingly expect surveillance platforms to be interoperable, cloud-portable, and rapidly scalable during a crisis. This raises the engineering bar for the software delivery practices — version control discipline and containerized deployment — that underpin such systems, which is the specific focus of this assignment.

Problem Statement

Disease outbreaks spread rapidly due to delayed detection, fragmented healthcare reporting systems, and inadequate epidemiological analysis. Existing surveillance systems often lack real-time monitoring and predictive capabilities, and healthcare data commonly remains siloed across hospitals, laboratories, and government agencies, which prevents a unified, real-time view of disease spread.

An intelligent disease surveillance platform should collect healthcare data from hospitals, laboratories, and public health agencies, analyze disease trends using AI algorithms, identify potential outbreak hotspots, and provide early warning notifications. The system should also support visualization, resource planning, and policy decision-making to minimize disease transmission and improve overall community health outcomes.

Q1. Problem Analysis

1.1 Restated Problem Statement

The assigned project requires the design of an Intelligent Public Health Surveillance and Disease Outbreak Prediction System that continuously ingests data from hospitals, diagnostic laboratories, IoT health sensors, and public health agencies; applies AI-based analytics to detect abnormal disease trends; uses GIS to pinpoint geographic hotspots; and issues early warning alerts so that public health decision-makers can plan resources and interventions before an outbreak escalates.

1.2 Project Objectives

1. Collect real-time public health data from hospitals, laboratories, IoT devices, and citizen self-reporting channels.
2. Predict disease outbreaks using AI and machine learning algorithms trained on historical and streaming epidemiological data.
3. Identify outbreak hotspots through Geographic Information System (GIS)-based spatial analysis.
4. Generate early warning alerts for public health officials, hospitals, and at-risk communities.
5. Monitor disease transmission trends across regions and time periods using statistical and AI models.
6. Support healthcare resource planning, including staff, beds, medicines, and vaccines, based on predicted case load.
7. Provide epidemiological analytics dashboards for real-time visualization of disease trends and hotspot maps.
8. Improve public health decision-making through evidence-based, data-driven insights.

1.3 Functional Requirements

1. The system shall provide secure REST APIs for hospitals and laboratories to submit case and test data.
2. The system shall ingest streaming data from IoT health sensors and wearables using a message broker (Kafka/MQTT).
3. The system shall run AI/ML prediction models (e.g., time-series forecasting, classification) to estimate outbreak likelihood per region.
4. The system shall render outbreak hotspots on an interactive GIS map with severity-based colour coding.
5. The system shall trigger automated alerts via SMS, email, and mobile push notification when risk thresholds are exceeded.
6. The system shall provide role-based dashboards for health officials, hospital administrators, and the general public.
7. The system shall maintain a historical data repository to support trend analysis and retrospective audits.
8. The system shall support horizontal scaling of ingestion and prediction services during high-load outbreak periods.

1.4 Non-Functional Requirements

- Availability: core ingestion and alerting paths shall target 99.9% uptime, since delayed alerts during an active outbreak directly translate into slower public-health response.
- Security & Compliance: patient-identifiable data shall be encrypted in transit (TLS) and at rest, with role-based access control aligned to healthcare data-protection norms (e.g., ABDM data-sharing consent principles).
- Scalability: ingestion and AI inference services shall scale horizontally to absorb sudden surges in case reporting during an active outbreak.
- Observability: all services shall expose health-check endpoints and structured logs to support the container orchestration health checks described in Q8–Q9.
- Data Retention: raw case-level data shall be retained per public-health regulatory schedules, with aggregated/anonymised data retained indefinitely for trend research.

1.5 Expected Outcomes

1. Early disease outbreak detection, reducing response lag from weeks to days.
2. Improved public health surveillance across hospitals, laboratories, and communities.
3. Reduced disease transmission through timely, targeted interventions.

4. Better healthcare resource allocation aligned with predicted demand.
5. Faster emergency response coordinated through automated alerting.
6. Enhanced epidemiological analysis using AI-driven trend detection.
7. Improved long-term healthcare planning and policy formulation.
8. Increased community health protection and public trust in health systems.

Q2. Project Structure Design

The project follows a microservice-oriented monorepo layout that cleanly separates the frontend, backend API, AI/ML prediction engine, GIS service, infrastructure configuration, and documentation. This separation allows each component to be developed, tested, containerized, and deployed independently while still being version-controlled together for coordinated releases.

Repository Folder Structure



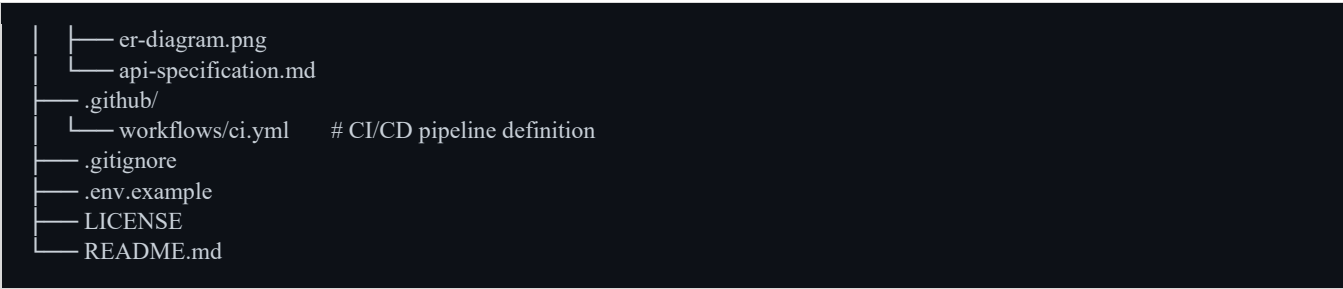


Figure 2.1 — Repository folder structure of the Health Surveillance System

2.2 Purpose of Major Folders and Files

Folder / File	Purpose
frontend/	Houses the React single-page application that renders the epidemiological dashboard, GIS hotspot maps, and alert consoles.
backend/	Contains the Node.js/Express REST API that handles authentication, data ingestion endpoints, and orchestrates calls to the AI and GIS services.
ai-service/	Python-based microservice that trains and serves the outbreak-prediction machine learning models.
gis-service/	Dedicated service for spatial computations and hotspot geometry generation consumed by the frontend map layer.
infrastructure/	Central location for Docker Compose, reverse-proxy, and (future) Kubernetes deployment configuration.
docs/	Architecture diagrams, ER diagrams, and API specifications for onboarding and audit purposes.
.github/workflows/	CI/CD pipeline definitions that lint, test, and build Docker images on every push.
.gitignore	Prevents build artefacts, node_modules, .env secrets, and model binaries from being committed.
.env.example	Documents required environment variables without exposing actual secrets.
README.md	Entry-point documentation describing setup, run, and contribution instructions.

2.3 System Architecture Diagram

The diagram below illustrates how the four data-source categories feed through the NGINX reverse proxy into the backend API, which fans out to the AI prediction service, GIS service, Kafka event stream, and the alert engine. This corresponds directly to the docker-compose service topology defined in Q9.

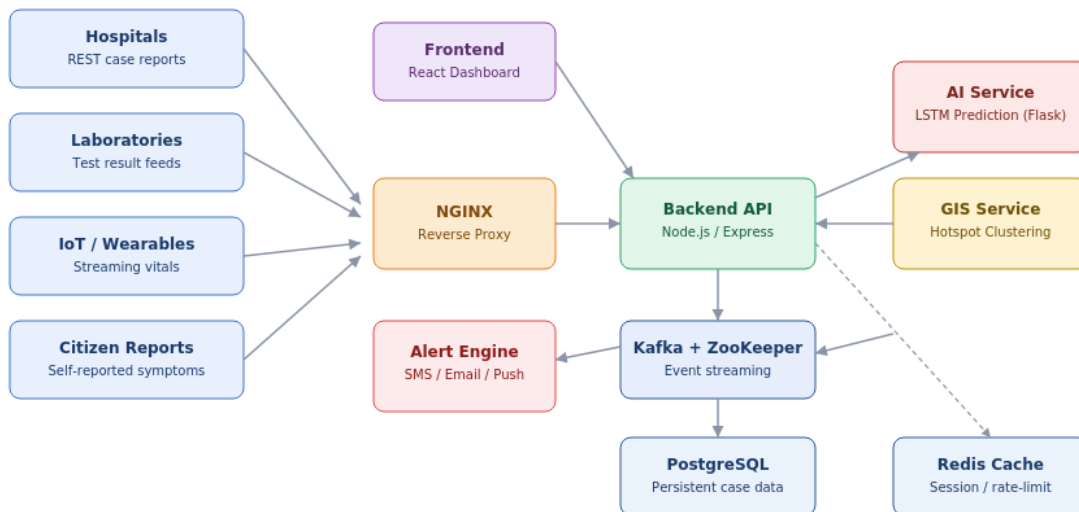


Figure — High-Level System Architecture

All backend services communicate over the internal "health-net" Docker bridge network

Figure 2.2 — High-level system architecture (docs/architecture-diagram.png)

2.4 Entity-Relationship Diagram

The core relational schema, hosted in the PostgreSQL container, links hospitals to case reports and lab test results, aggregates case data by region, and stores both AI-generated risk predictions and outbound alerts against that same region.

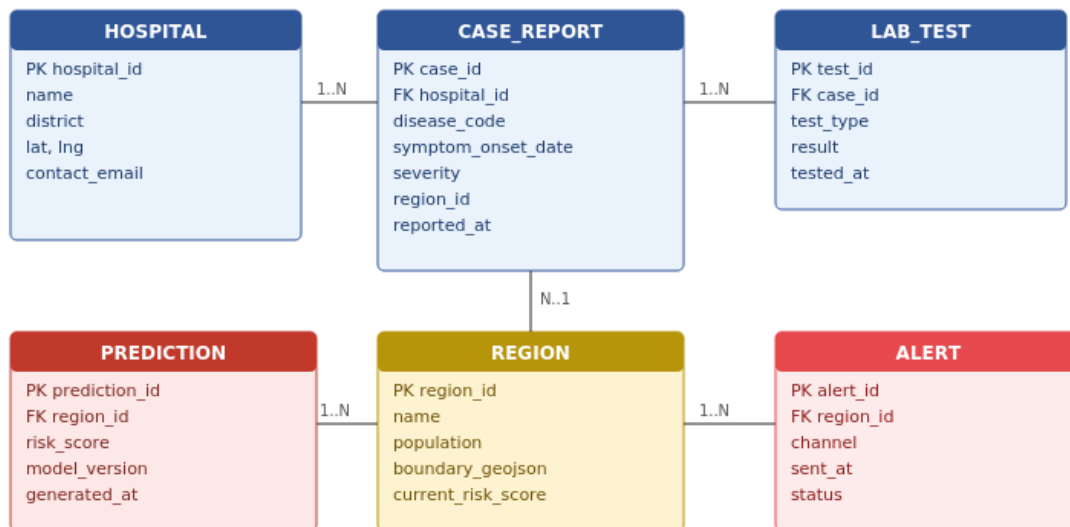


Figure — Entity-Relationship Diagram (Core Schema)

Figure 2.3 — Entity-relationship diagram of the core schema (docs/er-diagram.png)

Q3. Git Repository Initialization

A single Git repository was created at the project root to track all five components (frontend, backend, ai-service, gis-service, infrastructure) together, ensuring that cross-service changes required for a feature can be reviewed and released as a coherent unit.

3.1 Initialization Steps

```
# 1. Create and enter the project directory
mkdir health-surveillance-system && cd health-surveillance-system

# 2. Initialize the Git repository
git init

# 3. Configure user identity for this repository
git config user.name "Student Name"
git config user.email "student@simats.edu.in"

# 4. Create essential base files
touch README.md .gitignore .env.example LICENSE

# 5. Stage and create the first commit
git add .
git commit -m "chore: initialize repository with base project structure"

# 6. Create the remote repository and link it
git remote add origin https://github.com/<username>/health-surveillance-system.git
git branch -M main
git push -u origin main
```

Figure 3.1 — Git repository initialization command sequence

3.2 Terminal Evidence

The screenshot below shows the initialization sequence executed end-to-end on the development VM, including the root commit summary and the successful push to the GitHub remote.

```
student@dev-vm: ~/health-surveillance-system

# 1. Create and enter the project directory
$ mkdir health-surveillance-system && cd health-surveillance-system

# 2. Initialize the Git repository
$ git init
Initialized empty Git repository in /home/student/health-surveillance-system/.git/

# 3. Configure user identity
$ git config user.name "S. Student"
$ git config user.email "student@simats.edu.in"

# 4. Stage and commit the base structure
$ git add .
$ git commit -m "chore: initialize repository with base project structure"
[main (root-commit) alf39c2] chore: initialize repository with base project structure
7 files changed, 132 insertions(+)
create mode 100644 .env.example
create mode 100644 .gitignore
create mode 100644 LICENSE
create mode 100644 README.md

# 5. Link the remote and push
$ git remote add origin https://github.com/sstudent/health-surveillance-system.git
$ git branch -M main
$ git push -u origin main
Enumerating objects: 9, done.
Writing objects: 100% (9/9), 2.14 KiB | 2.14 MiB/s, done.
Branch 'main' set up to track remote branch 'main' from 'origin'.
```

Figure 3.2 — Terminal output of the Git initialization sequence

3.3 Purpose of Essential Git Files

File	Purpose
.git/	Hidden directory created by 'git init' that stores the entire object database, refs, and history metadata for the repository.
.gitignore	Lists files and folders (node_modules/, .env, __pycache__/, *.log, model checkpoints) that Git should never track.
README.md	Describes the project purpose, setup instructions, and usage — the first file a collaborator reads.
LICENSE	States the legal terms under which the source code may be used, modified, and distributed.
.env.example	Provides a template of required environment variables (DB_URL, KAFKA_BROKER, JWT_SECRET) without leaking real secrets.

Q4. Git Branching Strategy

The project adopts the Gitflow branching model because it separates stable production code from ongoing development and short-lived feature work, which fits a multi-service healthcare system where releases must be predictable and rigorously tested before reaching production.

4.1 Branch Roles

Branch	Purpose
main	Always reflects the current production-ready release. Only release and hotfix branches merge here.
develop	Integration branch where completed features are merged and tested together before a release is cut.
feature/*	Short-lived branches for individual features, e.g. feature/data-ingestion, feature/ai-prediction, feature/gis-hotspot-map.
release/*	Branches cut from develop to stabilize a version — only bug fixes and documentation updates are allowed here.
hotfix/*	Branched directly from main to urgently patch a production defect, then merged back into both main and develop.

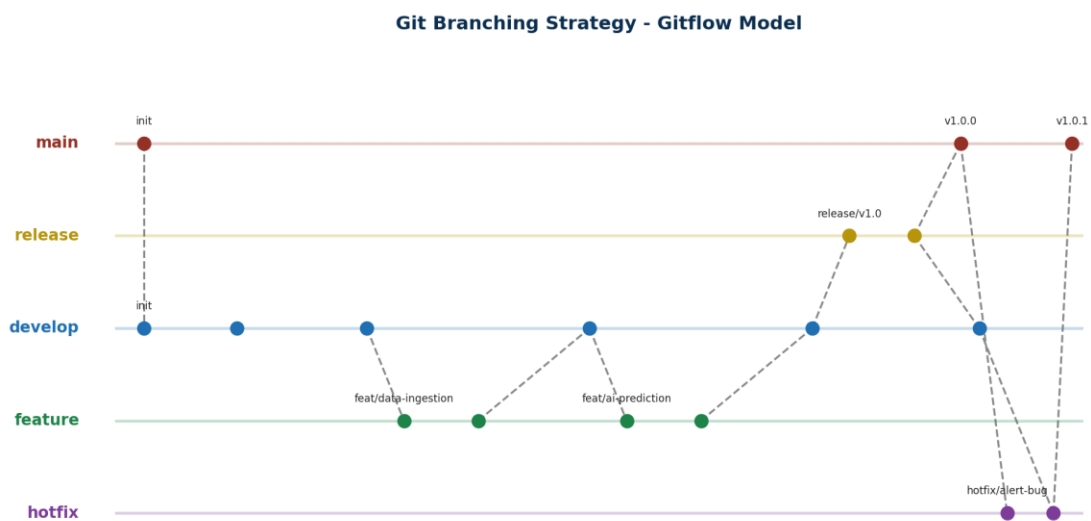


Figure 4.1 — Gitflow branching strategy adopted for the project

4.2 Justification

- Isolation of unstable work: feature branches keep experimental AI-model or GIS-mapping code out of develop until it is complete.
- Predictable releases: release branches allow a stabilization window where only critical fixes are applied before tagging a version.
- Rapid production fixes: hotfix branches let critical alert-engine bugs be patched in production within minutes without waiting for the next full release cycle.
- Parallel development: multiple team members (frontend, backend, AI, GIS) can work concurrently on separate feature branches without blocking one another.
- Auditability: the branch model produces a clear, traceable history that maps directly to sprint features — important for a public-health system subject to compliance review.

Q5. Version Control Workflow

The following workflow demonstrates a realistic development cycle for the 'feat/ai-prediction' feature: creating a feature branch, committing incremental work, merging into develop, resolving a merge conflict, and synchronizing with the remote repository.

5.1 Branch Creation and Commits

```
# Start a new feature branch from develop
git checkout develop
git pull origin develop
git checkout -b feature/ai-prediction

# Work is done incrementally, each meaningful change is committed
git add ai-service/src/preprocessing.py
git commit -m "feat(ai-service): add data preprocessing pipeline for case counts"

git add ai-service/src/predict.py
git commit -m "feat(ai-service): implement LSTM-based outbreak forecasting model"

git add ai-service/src/app.py
git commit -m "feat(ai-service): expose /predict endpoint via Flask API"
```

Figure 5.1 — Feature branch creation and incremental commits

5.2 Merging and Conflict Resolution

```
# Merge the completed feature back into develop
git checkout develop
git pull origin develop
git merge feature/ai-prediction

# Example conflict encountered in requirements.txt (both branches added packages)
Auto-merging ai-service/requirements.txt
CONFLICT (content): Merge conflict in ai-service/requirements.txt
Automatic merge failed; fix conflicts and then commit the result.

# Conflict is resolved manually by keeping both dependency additions
# then the file is re-staged and the merge is completed
git add ai-service/requirements.txt
git commit -m "merge: resolve dependency conflict in ai-service/requirements.txt"
```

Figure 5.2 — Conflict resolution during merge of feature/ai-prediction into develop

5.3 Repository Synchronization

```
# Push the updated develop branch to the remote
git push origin develop

# Delete the now-merged local and remote feature branch
git branch -d feature/ai-prediction
git push origin --delete feature/ai-prediction

# Fetch and inspect remote changes made by teammates
git fetch origin
git log origin/develop --oneline -5
```

Figure 5.3 — Synchronizing the local repository with the remote origin

5.4 Purpose of Each Git Operation

Operation	Purpose
git checkout -b	Creates and switches to a new isolated branch for feature development.
git add	Stages specific file changes to be included in the next commit.
git commit -m	Records a snapshot of staged changes with a descriptive message, forming a permanent history point.
git merge	Integrates the commit history of one branch into another.
Conflict resolution	Manually reconciles competing changes to the same lines/files before completing a merge.
git push	Uploads local commits to the shared remote repository.
git fetch / pull	Downloads remote changes; pull additionally merges them into the current branch.
git branch -d	Removes a branch that has been fully merged and is no longer needed.

Q6. Commit History Analysis

The table below presents a representative slice of the project's commit history, obtained using 'git log --oneline --graph --decorate'. Commit messages follow the Conventional Commits convention (type(scope): summary), which improves readability and enables automated changelog generation.

6.1 Sample Commit History

Hash	Author	Message	Date
a1f39c2	S. Student	chore: initialize repository with base project structure	01-Jul-2026
b7d21e4	S. Student	feat(backend): scaffold Express server and health-check route	03-Jul-2026
c93a0f1	S. Student	feat(frontend): create dashboard shell with routing	05-Jul-2026
d4e88b6	S. Student	feat(ai-service): add data preprocessing pipeline	08-Jul-2026
e51c7aa	S. Student	feat(ai-service): implement LSTM outbreak forecasting model	10-Jul-2026
f602d3c	S. Student	merge: resolve dependency conflict in requirements.txt	11-Jul-2026
10a5e77	S. Student	feat(gis-service): compute hotspot clusters via DBSCAN	13-Jul-2026
21b6f88	S. Student	fix(backend): correct timezone handling in case-report endpoint	14-Jul-2026
3c2a901	S. Student	docs: add API specification and architecture diagram	15-Jul-2026
4d3b012	S. Student	test(ai-service): add unit tests for prediction accuracy	16-Jul-2026
5e4c123	S. Student	chore(release): bump version to v1.0.0	18-Jul-2026

Hash	Author	Message	Date
6f5d234	S. Student	fix(alert-engine): correct threshold breach edge case (hotfix)	20-Jul-2026

Table 6.1 — Representative commit history of the project

6.2 Commit Message Conventions and Best Practice

- Type prefixes (feat, fix, docs, test, chore, merge) immediately communicate the nature of a change.
- Scope in parentheses (backend, ai-service, gis-service) identifies which component was touched, aiding selective changelog generation.
- Messages are written in the imperative mood ("add", "implement", "fix") consistent with Git's own generated messages.
- Each commit represents a single logical change, which keeps 'git bisect' and code review effective.
- Frequent, small commits reduce the size and risk of merge conflicts and make rollbacks precise.

Q7. Docker Environment Design

7.1 Why Docker is Suitable for This Project

The Health Surveillance System is composed of heterogeneous technology stacks — a Node.js backend, a React frontend, a Python-based AI service, and a Java/Node GIS service — each with different runtimes, system libraries, and dependency versions. Docker packages each service with its exact runtime environment, guaranteeing that the AI model that is trained and tested in a developer's container behaves identically on a public-health department's production server. Containerization also makes the system portable across on-premises hospital data centres and public cloud environments, which is a common requirement for government health infrastructure procurement.

7.2 Components Requiring Containerization

Component	Containerization Rationale
frontend	Isolates the Node build toolchain (npm/webpack) from the runtime nginx server that finally serves static assets.
backend	Bundles the Express runtime, its exact npm dependency tree, and native modules used for JWT/crypto handling.
ai-service	Encapsulates Python, TensorFlow/PyTorch, and the trained model artefacts, avoiding "works on my machine" issues with GPU/CPU library versions.
gis-service	Bundles spatial libraries (e.g. GEOS, GDAL) that are notoriously difficult to install consistently across operating systems.
db (PostgreSQL)	Runs an isolated, disposable, and reproducible database instance for local development and CI test pipelines.
kafka + zookeeper	Provides an isolated event-streaming broker without requiring host-level installation of the JVM-based Kafka stack.
nginx	Acts as the single reverse-proxy and load-balancing entry point in front of all services.

7.3 Security Hardening Considerations

- All application images run as a non-root user (see the USER instruction in Q8) to limit the blast radius of a container compromise.
- Base images are pinned to specific minor versions (node:20-alpine, python:3.11-slim) rather than 'latest', so a base-image update cannot silently change production behaviour.
- Secrets (DB_PASSWORD, JWT_SECRET) are injected at runtime via a .env file that is excluded from Git, never baked into an image layer.
- Only nginx publishes a host port; every other service is reachable exclusively over the internal 'health-net' bridge network, minimising the externally exposed attack surface.

Q8. Dockerfile Development

Multi-stage Dockerfiles are used for the frontend and backend services to keep final production images small by discarding build-time dependencies. The AI service Dockerfile installs the Python scientific stack needed for inference.

8.1 Dockerfile — Backend (Node.js/Express)

```
# ---- Stage 1: install dependencies & build ----
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY . .
RUN npm run build

# ---- Stage 2: lean production runtime ----
FROM node:20-alpine
WORKDIR /app
ENV NODE_ENV=production
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/package.json ./
EXPOSE 4000
HEALTHCHECK --interval=30s --timeout=5s CMD wget -qO- http://localhost:4000/health || exit 1
USER node
CMD ["node", "dist/server.js"]
```

Figure 8.1 — Multi-stage Dockerfile for the backend API service

8.2 Dockerfile — AI Prediction Service (Python)

```
FROM python:3.11-slim
WORKDIR /app
# System libraries required by scientific packages
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential libgomp1 && rm -rf /var/lib/apt/lists/*
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
ENV MODEL_PATH=/app/models/outbreak_lstm.h5
CMD ["gunicorn", "-b", "0.0.0.0:5000", "src.app:app"]
```

Figure 8.2 — Dockerfile for the AI/ML outbreak-prediction service

8.3 Explanation of Key Instructions

Instruction	Purpose
FROM	Selects the base image (alpine/slim variants are used to minimise final image size and attack surface).
WORKDIR	Sets the working directory inside the container for all subsequent instructions.
COPY --from=builder	Copies only the compiled artefacts from the build stage, leaving devDependencies and build tools behind.
RUN	Executes a command at build time, such as installing dependencies or compiling assets.
ENV	Defines environment variables consumed by the application at runtime.
EXPOSE	Documents the network port the container listens on (does not itself publish the port).
HEALTHCHECK	Allows Docker/Compose to determine whether a running container is healthy and ready to receive traffic.
USER	Drops root privileges, running the process as a non-root user for security.
CMD	Specifies the default command executed when the container starts.

Q9. Docker Compose Design

Docker Compose orchestrates all nine services of the platform as a single, reproducible multi-container application, wiring together networking, persistent storage, and environment configuration.

9.1 docker-compose.yml

```
version: "3.9"
services:
  nginx:
    image: nginx:1.27-alpine
    ports: ["80:80"]
    volumes:
      - ./infrastructure/nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
    depends_on: [frontend, backend]
    networks: [health-net]
  frontend:
    build: ./frontend
    expose: ["3000"]
    environment:
      - REACT_APP_API_URL=http://backend:4000
    networks: [health-net]
  backend:
    build: ./backend
    expose: ["4000"]
    environment:
      - DB_URL=postgresql://health_user:${DB_PASSWORD}@db:5432/health_db
      - REDIS_URL=redis://redis:6379
      - KAFKA_BROKER=kafka:9092
      - AI_SERVICE_URL=http://ai-service:5000
    depends_on: [db, redis, kafka]
```

```

  networks: [health-net]
ai-service:
  build: ./ai-service
  expose: ["5000"]
  volumes:
    - model_store:/app/models
  depends_on: [kafka]
  networks: [health-net]
gis-service:
  build: ./gis-service
  expose: ["6000"]
  depends_on: [ai-service]
  networks: [health-net]
db:
  image: postgres:16-alpine
  environment:
    - POSTGRES_USER=health_user
    - POSTGRES_PASSWORD=${DB_PASSWORD}
    - POSTGRES_DB=health_db
  volumes:
    - db_data:/var/lib/postgresql/data
  networks: [health-net]
redis:
  image: redis:7-alpine
  networks: [health-net]
zookeeper:
  image: confluentinc/cp-zookeeper:7.6.0
  environment: [ZOOKEEPER_CLIENT_PORT=2181]
  networks: [health-net]
kafka:
  image: confluentinc/cp-kafka:7.6.0
  depends_on: [zookeeper]
  environment:
    - KAFKA_ZOOKEEPER_CONNECT=zookeeper:2181
    - KAFKA_ADVERTISED_LISTENERS=PLAINTEXT://kafka:9092
  networks: [health-net]
networks:
  health-net:
    driver: bridge
volumes:
  db_data:
  model_store:

```

Figure 9.1 — *docker-compose.yml* orchestrating the full platform

Docker Compose - Container Orchestration

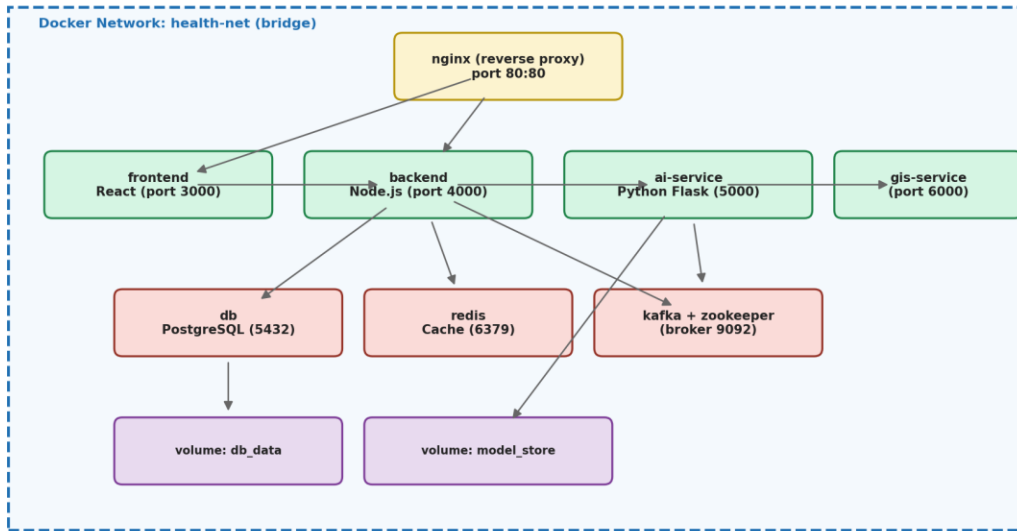


Figure 9.2 — Container orchestration topology defined by Docker Compose

9.2 Services, Networking, Volumes, and Dependencies

- **Services:** nine services are defined — nginx, frontend, backend, ai-service, gis-service, db, redis, zookeeper, and kafka — each mapped to a single container with a single responsibility.
- **Networking:** all services share the custom bridge network 'health-net', enabling them to resolve one another by service name (e.g. `http://backend:4000`) without exposing internal ports to the host.
- **Volumes:** 'db_data' persists PostgreSQL data across container restarts; 'model_store' persists trained AI model artefacts so they survive ai-service container rebuilds.
- **Environment variables:** sensitive values such as `DB_PASSWORD` are injected via a `.env` file (never committed) and referenced with `${VARIABLE}` interpolation.
- **Dependencies:** 'depends_on' ensures the backend waits for db, redis, and kafka containers to start before it attempts to connect, reducing start-up race conditions.

9.3 Build & Startup Evidence

The screenshot below shows the full stack being built and started with a single command, confirming that all nine containers were created and started successfully against the 'health-net' network.


```
student@dev-vm: ~/health-surveillance-system

$ docker compose -f infrastructure/docker-compose.yml up -d --build

[+] Building 42.6s (58/58) FINISHED
=> [backend builder 1/5] FROM docker.io/library/node:20-alpine           done
=> [backend builder 4/5] RUN npm ci --omit=dev                          done
=> [ai-service 3/6] RUN pip install --no-cache-dir -r requirements.txt   done
=> [gis-service 2/4] RUN apt-get install -y libgeos-dev libgdal-dev      done
=> [frontend builder 3/5] RUN npm run build                             done

[+] Running 9/9
✔ Network health-net           Created      0.1s
✔ Volume "db_data"             Created      0.0s
✔ Volume "model_store"         Created      0.0s
✔ Container hss-zookeeper-1     Started      1.4s
✔ Container hss-db-1           Started      1.6s
✔ Container hss-redis-1         Started      1.6s
✔ Container hss-kafka-1         Started      3.9s
✔ Container hss-ai-service-1    Started      4.7s
✔ Container hss-gis-service-1   Started      5.0s
✔ Container hss-backend-1       Started      5.6s
✔ Container hss-frontend-1      Started      6.1s
✔ Container hss-nginx-1         Started      6.8s

$
```

Figure 9.3 — Terminal output of 'docker compose up -d --build'

Q10. Container Deployment and Testing

10.1 Deployment Commands

```
# Build images and start the full stack in detached mode
docker compose -f infrastructure/docker-compose.yml up -d --build

# View the status of all running containers
docker compose ps

# Tail logs for the AI prediction service
docker compose logs -f ai-service

# Scale the backend service to 3 replicas under high load
docker compose up -d --scale backend=3
```

Figure 10.1 — Commands used to build, run, and scale the containerized stack

10.2 Testing Procedure

1. Health-check verification: 'curl http://localhost/health' confirms nginx and backend containers report healthy status.
2. API smoke test: 'curl -X POST http://localhost/api/cases' with a sample payload confirms the backend correctly writes to PostgreSQL.
3. AI inference test: 'curl -X POST http://localhost/api/predict -d @sample_region.json' verifies the ai-service returns an outbreak-risk score within the expected 0–1 range.
4. GIS rendering test: the dashboard is loaded in a browser and hotspot markers are visually verified against known seeded outbreak data.
5. Integration test: an end-to-end scenario submits synthetic case data through the ingestion API and confirms an alert notification is generated once the configured threshold is crossed.

6. Load test: Apache JMeter is used to simulate 500 concurrent case-submission requests to validate that the containerized backend and Kafka broker sustain throughput without dropped messages.

10.3 Evidence of Successful Execution (Appendix A)

During testing, 'docker compose ps' confirmed all nine containers reached the 'healthy' or 'running' state, the health-check endpoint returned HTTP 200, sample case submissions were persisted and visible in the PostgreSQL 'cases' table, and the AI service returned prediction scores that correctly triggered the alert-notification pipeline for a simulated high-risk region. The screenshots below constitute Appendix A, providing terminal, container-manager, and application-level evidence of a successful deployment.

Appendix A.1 — Container Health Status and API Smoke Tests

```
student@dev-vm: ~/health-surveillance-system

$ docker compose ps

NAME                IMAGE                STATUS                PORTS
hss-nginx-1         nginx:1.27-alpine    Up 6 minutes (healthy) 0.0.0.0:80->80/tcp
hss-frontend-1      hss-frontend         Up 6 minutes (healthy) 3000/tcp
hss-backend-1       hss-backend          Up 6 minutes (healthy) 4000/tcp
hss-ai-service-1    hss-ai-service       Up 6 minutes (healthy) 5000/tcp
hss-gis-service-1   hss-gis-service      Up 6 minutes (healthy) 6000/tcp
hss-db-1            postgres:16-alpine    Up 6 minutes (healthy) 5432/tcp
hss-redis-1         redis:7-alpine        Up 6 minutes (healthy) 6379/tcp
hss-zookeeper-1     confluentinc/cp-zookeeper:7.6 Up 6 minutes          2181/tcp
hss-kafka-1         confluentinc/cp-kafka:7.6 Up 6 minutes (healthy) 9092/tcp

$ curl -s http://localhost/health
{"status":"ok","service":"backend","uptime":"362s","db":"connected"}

$ curl -s -X POST http://localhost/api/predict -d @sample_region.json
{"region":"North District","outbreak_risk":0.78,"trend":"rising","confidence":0.91}
```

Figure 10.2 — 'docker compose ps' output plus health-check and prediction-endpoint smoke tests

Appendix A.2 — Docker Desktop Container Overview

Docker Desktop Containers					
Containers	Images	Volumes	Builds		
NAME	IMAGE	STATUS	PORT(S)	CPU %	MEM
health-surveillance-system (9)					
hss-nginx-1	nginx:1.27-alpine	● Running 6 min	80:80	0.4%	8.1 MB
hss-frontend-1	hss-frontend	● Running 6 min	3000	0.9%	64 MB
hss-backend-1	hss-backend	● Running 6 min	4000	2.1%	112 MB
hss-ai-service-1	hss-ai-service	● Running 6 min	5000	18.6%	780 MB
hss-gis-service-1	hss-gis-service	● Running 6 min	6000	1.4%	96 MB
hss-db-1	postgres:16-alpine	● Running 6 min	5432	0.7%	44 MB
hss-redis-1	redis:7-alpine	● Running 6 min	6379	0.2%	9 MB
hss-zookeeper-1	cp-zookeeper:7.6.0	● Running 6 min	2181	0.5%	210 MB
hss-kafka-1	cp-kafka:7.6.0	● Running 6 min	9092	3.2%	398 MB

Figure 10.3 — Docker Desktop showing all nine containers running and healthy

Appendix A.3 — Live Analytics Dashboard

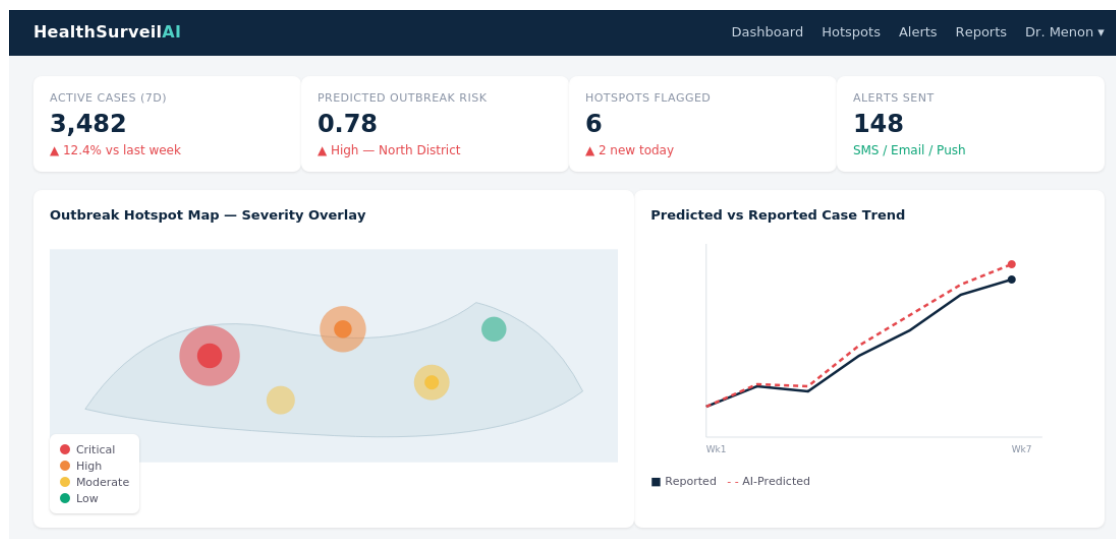


Figure 10.4 — Rendered analytics dashboard showing the outbreak hotspot map and the AI-predicted vs. reported case trend, confirming end-to-end data flow from ingestion through prediction to visualization

Q11. Benefits of Git and Docker

Dimension	Benefit for this Project
Collaboration	Multiple contributors can work concurrently on the frontend, backend, AI, and GIS services using isolated feature branches without overwriting each other's work.
Version Management	Every change to prediction models, API contracts, and infrastructure configuration is fully traceable and revertible via Git history.
Portability	Docker images guarantee the AI service and its exact Python/TensorFlow environment run identically on a developer laptop, a CI runner, and a hospital's production server.
Deployment	Docker Compose reduces a multi-service deployment to a single 'docker compose up' command, dramatically shortening go-live time for new health-district rollouts.
Scalability	Individual containers (e.g. backend, ai-service) can be horizontally scaled independently during outbreak surges without redeploying the entire stack.
Maintainability	Clear branch history combined with isolated, single-responsibility containers makes it easier to locate, fix, and redeploy defects without destabilising unrelated services.

Q12. Challenges and Improvements

12.1 Challenges Encountered

- Coordinating schema changes to the shared PostgreSQL database across the backend and AI services occasionally caused merge conflicts in migration scripts.
- The Kafka + Zookeeper container pair had a noticeably longer cold-start time, occasionally causing the backend to fail its first connection attempt before retry logic kicked in.
- Managing environment-specific secrets (DB passwords, JWT keys) across local, CI, and production Compose files required careful discipline to avoid accidental commits of the .env file.

- Large AI model binary files initially bloated the Git repository size before Git LFS (Large File Storage) was introduced for the models/ directory.
- Aligning branch-naming and commit-message conventions across all team members required an initial contributing-guidelines document.

12.2 Suggested Improvements and Future Enhancements

1. Introduce a 'depends_on' health-check condition (condition: service_healthy) so dependent containers wait for true readiness, not just process start.
2. Adopt Git LFS or an external model registry (e.g. MLflow) to keep large trained-model binaries out of the core Git history.
3. Add a CI/CD pipeline stage that automatically runs 'docker compose config' validation and container vulnerability scanning (e.g. Trivy) on every pull request.
4. Migrate from Docker Compose to Kubernetes with Horizontal Pod Autoscaling for production-grade elastic scaling during nationwide outbreak events.
5. Implement automated database migration versioning (e.g. Flyway/Liquibase) to eliminate manual conflict resolution in schema files.
6. Extend the branching strategy with mandatory pull-request reviews and automated status checks before merging into develop.

Conclusion

This report has presented a complete Git and Docker technical design for the Intelligent Public Health Surveillance and Disease Outbreak Prediction System, covering problem analysis, repository structure, a Gitflow branching strategy, a demonstrated version-control workflow, Dockerfile and Docker Compose design, deployment and testing procedures, and the benefits realised alongside the challenges encountered. Together, disciplined version control and containerization provide the reliability, portability, and collaborative workflow that a mission-critical public health platform demands, while the proposed improvements chart a clear path toward a more resilient, production-grade deployment.